

[강의 노트] Java 제어문 (조건문 & 반복문)

1. 조건문을 활용한 분기 처리

조건문은 주어진 조건식의 결과(`true` / `false`)에 따라 프로그램의 실행 흐름을 바꾸는 문법입니다.

1) if - else 문

가장 대표적인 조건문으로, 조건의 참/거짓 여부에 따라 실행할 블록(`{ }`)을 결정합니다.

- 구조:

```
int score = 85;

if (score >= 90) {
    System.out.println("A 학점");
} else if (score >= 80) { // 위 조건이 거짓일 때 새로운 조건
    확인
    System.out.println("B 학점");
} else { // 모든 조건이 거짓일 때 실행
    System.out.println("C 학점 이하");
}
```

2) switch - case 문

특정 변수의 값에 따라 실행할 위치를 선택하는 조건문입니다. 범위를 비교하는 `if` 문과 달리, 딱 떨어지는 값(정수, 문자, 문자열)을 비교할 때 가독성이 좋습니다.

- 구조:

```
int rank = 2;

switch (rank) {
    case 1:
        System.out.println("금메달");
        break;
    case 2:
```

```

        System.out.println("은메달");
        break; // switch 문을 탈출하게 만듦
    case 3:
        System.out.println("동메달");
        break;
    default: // 일치하는 case가 없을 때 실행 (if문의 else 역할)
        System.out.println("참가상");
}

```

⚠ **주의 (break의 중요성):** case 끝에 break; 를 생략하면 다음 case의 코드까지 연달아 실행되는 **Fall-through** 현상이 발생하므로 주의해야 합니다.

2. 반복문과 루프 제어

특정 명령을 여러 번 반복해서 실행하고자 할 때 사용합니다.

1) for 문 (반복 횟수가 명확할 때)

초기화식, 조건식, 증감식을 한 줄에 모아놓아 구조가 직관적입니다. 주로 반복 횟수를 미리 알고 있을 때 사용합니다.

- 구조

```

// 초기화식; 조건식; 증감식
for (int i = 1; i <= 5; i++) {
    System.out.println(i + "번째 출력");
}

```

2) while 문 (조건에 따라 반복할 때)

조건식이 참(true)인 동안 블록 내부를 무한히 반복합니다. 반복 횟수가 불명확하고, 특정 조건이 만족될 때까지 돌려야 할 때 주로 씁니다.

- 구조

```

int count = 1; // 초기화

while (count <= 5) { // 조건식
    System.out.println(count);
}

```

```
count++; // 증감식 (생략하면 무한 루프에 빠짐!)
}
```

3) 루프 제어 명령어 (**break** vs **continue**)

- **break** : 반복문을 즉시 종료하고 빠져나갑니다.
- **continue** : 반복문의 나머지 코드를 건너뛰고 다음 반복(증감식 또는 조건식)으로 바로 넘어갑니다.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue; // 3일 때는 출력을 건너뛰고 i++로 이동
    }
    if (i == 5) {
        break; // 5일 때는 반복문을 완전히 종료
    }
    System.out.println(i);
}
// 출력 결과: 1, 2, 4
```

3. 조건/반복문 활용 실전 연습

1) 구구단 2단~9단 출력하기 (이중 for 문)

바깥쪽 **for** 문이 한 번 돌 때, 안쪽 **for** 문은 처음부터 끝까지 도는 원리를 이용합니다.

```
public class GuguDan{
    public static void main(String[] args){
        for (int i = 2; i <= 9; i++) { // 단 (2단 ~ 9단)
            System.out.println("--- " + i + "단 ---");
            for (int j = 1; j <= 9; j++) { // 곱하는 수 (1 ~
9)
                System.out.println(i + " x " + j + " = " +
(i * j));
            }
        }
    }
}
```

```
}  
}
```

실습 1단계: 별로 그리는 사각형 (기초 체력 기르기)

중첩 반복문으로 가장 먼저 만들어보는 기초 코스, 가로 5칸, 세로 3칸 크기의 별(\$*\$) 사각형 만들기입니다.

문제

아래와 같이 출력되는 프로그램을 작성해 보세요.

```
*****  
*****  
*****
```

▼ 💡 풀이 및 설명

- **바깥쪽 for 문:** 줄(세로)을 담당합니다. 총 3줄이 필요하니 **3번** 반복합니다.
- **안쪽 for 문:** 한 줄에 찍힐 별(가로)의 개수를 담당합니다. 한 줄당 별이 5개 필요하니 **5번** 반복합니다.
- **줄 바꿈:** 안쪽 루프가 끝나서 한 줄에 별 5개를 다 찍으면, 바깥쪽 루프가 끝나기 전에 `System.out.println()` 으로 줄 바꿈을 해줍니다.

```
public class StarRectangle{  
    public static void main(String[] args){  
        for (int i = 1; i <= 3; i++) { // 1. 세로 3줄  
            제어  
                for (int j = 1; j <= 5; j++) { // 2. 가로  
                    별 5개 제어  
                        System.out.print("*"); // 줄 바꿈 없이  
                            옆으로 쭉 찍기  
                                }  
                                    System.out.println(); // 3. 한 줄 다 찍었으  
                                        니 줄 바꿈!  
                                            }  
                                                }  
                                                    }  
                                                        }
```

실습 2단계: 직각삼각형 별 찍기 (바깥쪽 변수 활용하기)

단순한 사각형이 아닌, 줄이 바뀔 때마다 별의 개수가 늘어나는 구조입니다. 안쪽 `for` 문의 조건식에 바깥쪽 변수 `i` 를 녹여내는 것이 핵심입니다.

문제

아래와 같이 출력되는 프로그램을 작성해 보세요.

```
*
**
***
****
*****
```

▼ 💡 풀이 및 설명

- 첫 번째 줄($i = 1$)일 때는 별이 **1개** 필요합니다.
- 두 번째 줄($i = 2$)일 때는 별이 **2개** 필요합니다.
- 다섯 번째 줄($i = 5$)일 때는 별이 **5개** 필요합니다.
- 즉, 안쪽 루프가 도는 횟수(j 의 현재 값)가 바깥쪽 루프의 현재 횟수(i)와 같아야 합니다. 따라서 조건식을 `j <= i` 로 설정합니다.

```
public class StarTriangle{
    public static void main(String[] args){
        for (int i = 1; i <= 5; i++) { // 총 5줄 반복
            for (int j = 1; j <= i; j++) { // j가 i 이하일
                // 별 찍기
                System.out.print("*");
            }
            System.out.println(); // 줄 바꿈
        }
    }
}
```

실습 3단계: 헬스장 3분할 루틴 시뮬레이터 (실무형 응용)

숫자와 별 찍기에서 벗어나 실생활의 개념에 대입해 보면 이중 `for` 문이 더 친근하게 느껴집니다. 3일간 운동을 하는데, 매일 3가지 세트를 수행하는 프로그램을 만들어 봅시다.

문제

아래와 같이 출력되는 프로그램을 작성해 보세요.

```
=== [1일차 운동 시작] ===  
1세트 수행 완료!  
2세트 수행 완료!  
3세트 수행 완료!  
오늘 운동 끝!  
  
=== [2일차 운동 시작] ===  
1세트 수행 완료!  
...  
=== [3일차 운동 시작] ===  
...  
오늘 운동 끝!
```

▼ 💡 풀이 및 설명

- **바깥쪽 for 문:** 전체 일정인 1일차부터 3일차까지를 제어합니다 (\$i\$가 1부터 3까지).
- **안쪽 for 문:** 매일 소화해야 하는 운동 세트수(1세트~3세트)를 제어합니다 (\$j\$가 1부터 3까지).
- 하루(바깥 루프 1회)가 시작할 때 "운동 시작"을 알리고, 3세트(안쪽 루프 3회)를 마칠 때마다 "오늘 운동 끝!"을 출력합니다.

```
public class WorkoutRoutine{
    public static void main(String[] args){
        for (int day = 1; day <= 3; day++) { // 바깥쪽: 날짜 제어
            System.out.println("=== [" + day + "일차 운동 시작] ===");

            for (int set = 1; set <= 3; set++) { // 안쪽: 세트 수 제어
                System.out.println(set + "세트 수행 완료!");
            }

            System.out.println("오늘 운동 끝!\n"); // 하루 루틴 종료 후 빈 줄 추가
        }
    }
}
```